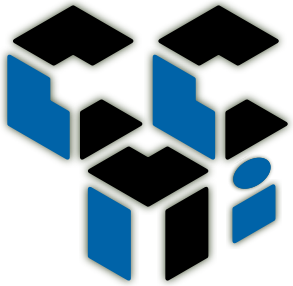
Technologie RDF a OWL





FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

BI-KOM: Konceptuální modelování #8

Ing. Marek Suchánek, Ph.D. et PhD (marek.suchanek@fit.cvut.cz)

Obsah přednášky

1. Motivace: proč RDF/OWL v praxi?
2. Základní myšlenky: triplety, grafy, otevřený svět
3. RDF, RDFS a jejich formáty
4. Ontologie s OWL
5. Dotazovací jazyk SPARQL
6. Validace dat pomocí SHACL
7. A co s tím dál?
    - Grafové databáze, *triple stores*
    - WIDOCO, WebVOWL, Protégé
    - Schema.org
    - FAIR data
    - Nanopublikace

Motivace: data v silech

- Představte si univerzitu: studijní systém, knihovna, personální systém, databáze publikací, rezervace místností
- Každý z nich má **vlastní databázi**, **vlastní schéma** a **vlastní identifikátor** pro *tutéž osobu*
  - `student_id = 123456`, `prukaz_knihovny = CVUT-9981`, `osobni_cislo = E-4472`
  - `orcid = 0000-0001-7525-9218`, `email = ema@fit.cvut.cz`
- Jednoduchá otázka: *Kteří vyučující mých předmětů publikovali článek s mým vedoucím práce?*
- Odpověď je rozprostřená přes tři systémy a žádný z nich ji sám nezná
- Propojení n systémů po dvojicích znamená až $\frac{n(n-1)}{2}$ integrací, které se rozbíjejí s každou změnou schématu
- Nechybí nám data, ale **sdílený a explicitně vyjádřený význam** dat

Motivace: RDF používáte každý den

- Vložte odkaz do Messengeru, Slacku nebo Teams a objeví se **náhledová karta**
- Ta se generuje z metadat Open Graph, která jsou definovaná nad **RDFa**

```
<meta property="og:title" content="Technologie RDF a OWL" />
<meta property="og:type" content="article" />
<meta property="og:url" content="https://kom.fit.cvut.cz/prednasky/08" />
<meta property="og:image" content="https://kom.fit.cvut.cz/img/rdf.png" />
```

- Každé takové `<meta>` je jen **triplet**: *(tato stránka, og:title, "Technologie RDF a OWL")*
- Zhruba polovina procházených webových stránek nese strukturovaná data tohoto typu (Web Data Commons)

Motivace: vyhledávače čtou vaše data

- Vyhledávače zpracovávají anotace Schema.org, obvykle jako **JSON-LD** (= RDF)

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Recipe",
  "name": "Masové kuličky od Emy",
  "author": {"@type": "Person", "name": "Ema"},
  "cookTime": "PT30M",
  "recipeIngredient": ["300 g hovězího", "1 vejce"]
}
</script>
```

- Přínos: *rich results* (hodnocení, cena, doba přípravy), Knowledge Panel, Dataset Search
- Jeden a tentýž soubor je zároveň **stránkou pro lidi** i **daty pro stroje**

Motivace: RDF v praxi

- **Znalostní grafy**: Wikidata (100+ milionů položek, živí Wikipedii i hlasové asistenty), DBpedia, Google Knowledge Graph
- **Veřejná správa**: data.gov.cz (Národní katalog otevřených dat, DCAT-AP), Sémantický slovník pojmů, data.europa.eu
- **Přírodní vědy**: UniProt (miliardy tripletů s veřejným SPARQL endpointem), Bio2RDF, PubChem
- **Zdravotnictví**: terminologie SNOMED CT je distribuována jako ontologie v OWL 2 EL
- **Kulturní dědictví**: Europeana (EDM), knihovní katalogy (BIBFRAME)
- **Publikování a průmysl**: BBC, Springer Nature SciGraph, podnikové a inženýrské znalostní grafy

Motivace: co RDF přináší

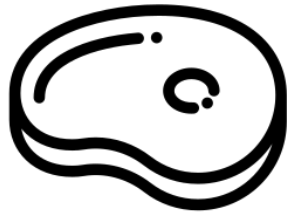
- **Globální identifikátory** = totéž URI znamená totéž v každém datasetu na světě
- **Integrace slučováním** = spojení dvou RDF grafů je *sjednocení tripletů*, bez mapování schémat
- **Schéma až potom** = přidání vlastnosti nevyžaduje `ALTER TABLE` ani migraci dat
- **Sebepopisná data** = slovník je na Webu na svém vlastním URI (RDFS/OWL)
- **Odvozování** = fakta se odvozují, neukládají (*ancestor* je tranzitivní, každá *Žena* je *Osoba*)
- **Jediný dotazovací jazyk** na to všechno = SPARQL, a to i napříč vzdálenými endpointy (*federated queries*)

Pozn.: Nic není zadarmo — cenou je upovídanost, výkon a vyzrálost nástrojů (viz konec přednášky).

Základní myšlenka: jednotka informace

- Jaká je nejmenší jednotka informace?
  - Z teorie informace **1 bit** = pravdivostní hodnota *Ano* / *Ne*
  - Bit je ale sám o sobě k ničemu, *Ano* / *Ne* náleží k nějakému tvrzení
- Jaká je nejjednodušší a nejuniverzálnější forma tvrzení?



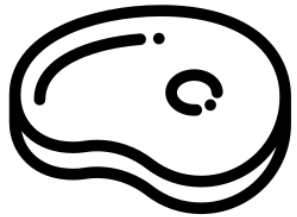


Ema mele maso

Semantic Triples (*triplety*)

- Nejjednodušší a nejuniverzálnější forma tvrzení = trojice / **triple**:

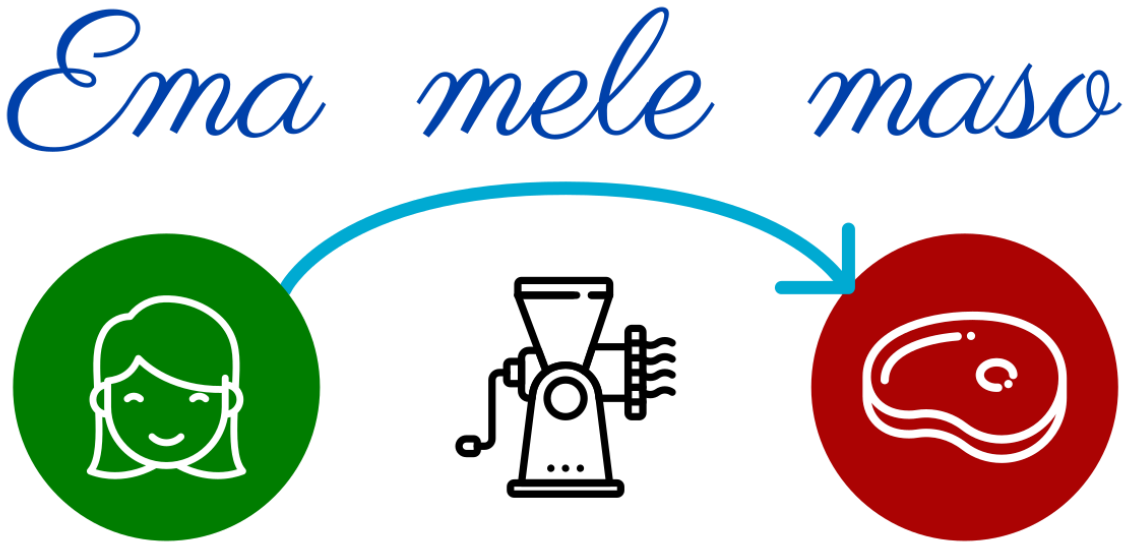




Ema	mele	maso
podmět	přísudek	předmět
subject	predicate	object

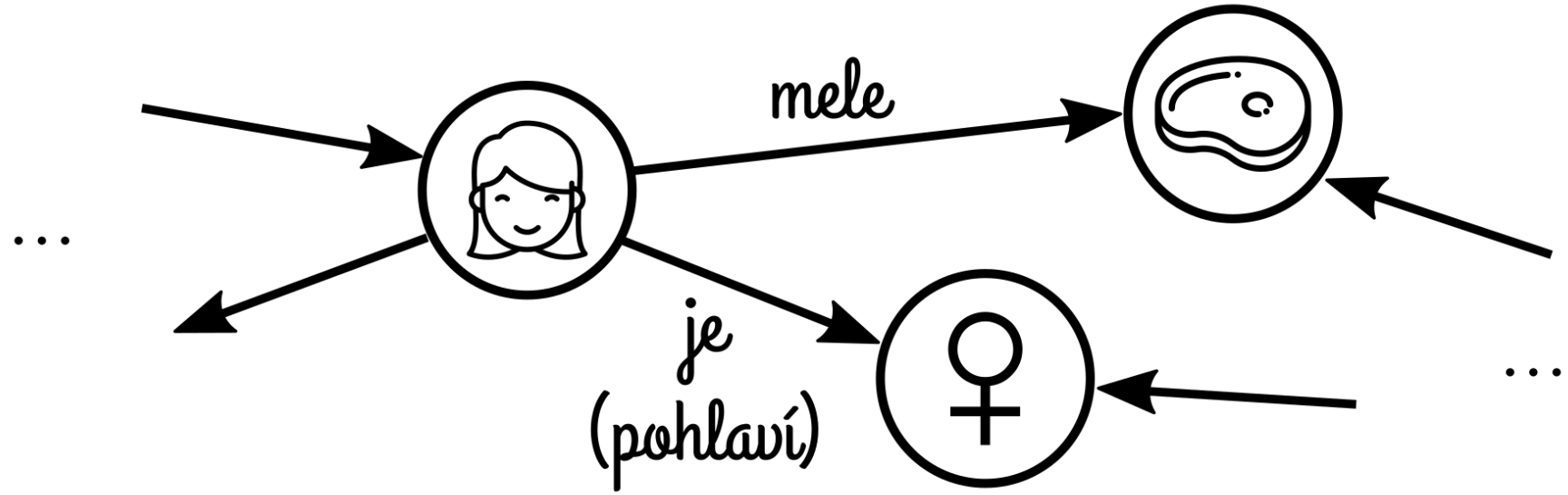
Tvrzení jako graf

- Taková tvrzení lze zakreslovat jako grafy:
  - **subject** a **object** tvoří uzly
  - **predicate** tvoří orientované hrany mezi uzly



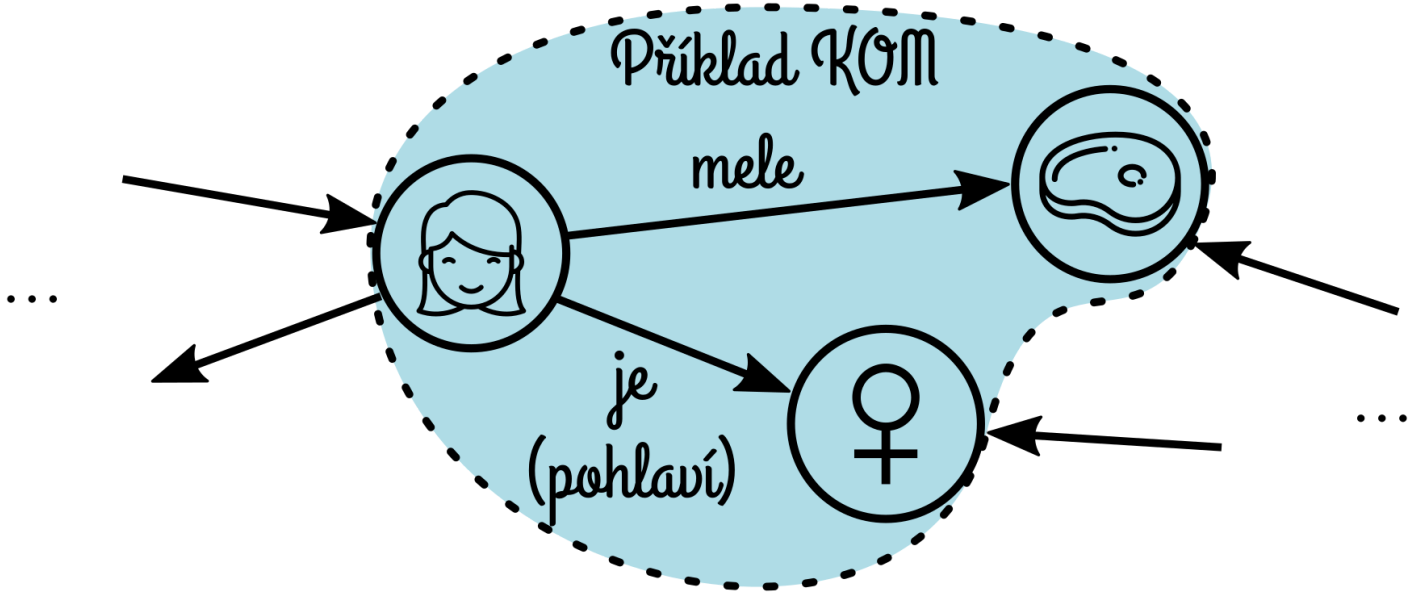
Propojování tripletů

- Tvrzení lze pak propojovat, hledat souvislosti a odvozovat nová
  - *Ema mele maso*, *Ema je žena*, ...
  - Existuje *žena*, která *mele maso*?
- Lze *importovat* i již existující další tvrzení a propojit je



Triplety v kontextech = quads

- V některých případech je potřeba specifikovat, k čemu se tvrzení vztahují = **kontext**
- K tomuto účelu slouží pojmenovaný graf (**named graph**)
- Poté se nebavíme o *trojicích*, ale o *čtveřicích* (**quads**)
- Mimo jiné to umožňuje dělat další tvrzení i o celém grafu



Uzavřený nebo otevřený svět

- Máme-li množinu tvrzení, je otázkou, co platí o nevyřčených tvrzeních
  - Platí například *Ema studuje na ČVUT*?
- **Open-world assumption** (OWA)
  - To, že jsme tvrzení *Ema studuje ČVUT* nevyřkli **neznamená**, že není pravdivé
  - Nelze rozhodnout zda je nebo není pravdivé, dokud jej explicitně nevyřkneme nebo jej nelze odvodit z jiných existujících tvrzení (například *Ema studuje ZŠ* a nelze studovat zároveň *ZŠ* a *ČVUT*)
  - Typické pro sémantický web (RDF, OWL, ...) ale i reálný svět

Uzavřený nebo otevřený svět

- Máme-li množinu tvrzení, je otázkou, co platí o nevyřčených tvrzeních
  - Platí například *Ema studuje na ČVUT*?
- **Closed-world assumption** (CWA)
  - To, že jsme tvrzení *Ema studuje na ČVUT* nevyřkli **znamená**, že není pravdivé
  - Zde musíme vyřknout vše, co platí, protože všechno ostatní implicitně není pravdivé
  - Typické pro deklarativní programování, databáze, ... a příslušné (i konceptuální) modely = co v nich není zachyceno, neexistuje

---

- Příklad: *Kouření způsobuje rakovinu*. CWA? OWA?

RDF: Resource Description Framework

- **Resource Description Framework** (systém popisu zdrojů)
  - Specifikován W3C původně jako model *metadat*
  - Stojí na myšlence popisu zdrojů pomocí tripletů
  - Hlavní komponentou *sémantického webu*
- **Zdroj** = cokoliv unikátně identifikováno pomocí URI
  - *Ema* = `http://example.com/bi-kom/rdf/example#ema`
  - Na dané adrese nemusí nutně existovat a být volně dosažitelný obsah resp. samotný zdroj, ale je to vhodné (viz konec přednášky)
  - *Marek* = `https://orcid.org/0000-0001-7525-9218`

URL, URN, URI, IRI

- **URL** = specifikuje umístění zdrojů
  - `http://example.com/bi-kom/rdf/example`
- **URN** = specifikuje samotný zdroj ale ne cestu k jeho dosažení
  - `example.com/bi-kom/rdf/example#ema`
- **URI** = spojení URL a URN pro přesnou specifikaci zdroje
  - `http://example.com/bi-kom/rdf/example#ema`
- **IRI** = rozšiřuje povolené znaky dle RFC3987 nad rámec URI (které dovoluje jen velmi omezenou sadu znaků + *percent-encoding* viz RFC3986)
  - `https://en.wiktionary.org/wiki/Ῥόδος` (IRI)
  - `https://en.wiktionary.org/wiki/%E1%BF%AC%CF%8C%CE%B4%CE%BF%CF%82` (URI)

Prefix, prázdné uzly, literály

- V zápise RDF se často využívá prefixů pro následné zkracování URI
  - Prefix `bi-kom-rdf:` bude reprezentovat `http://example.com/bi-kom/rdf/example#`
  - Potom můžeme o *Ema* psát jako `bi-kom-rdf:ema`
- Někdy chceme dělat ale tvrzení o nějakém *pomocném* zdroji
  - Chceme říct jaké maso Ema mele, ale nemáme URI daného masa
  - `bi-kom-rdf:ema bi-kom-rdf:mele _:maso`
  - `_:maso bi-kom-rdf:jeMaso druhy-masa:hovězí`
  - `_:maso měření:váží-gramů "300"^^xs:integer` (`"300"` je literál typu `xs:integer`)
  - `bi-kom-rdf:ema bi-kom-rdf:máJméno "Ema"` (`"Ema"` je literál typu `xs:string`)

(V dalších příkladech již použijeme angličtinu...)

Formáty RDF

- RDF popisuje jenom obecný popis zdrojů pomocí trojic
- Tudíž je specifikace RDF nezávislá na syntaxi
- Na druhou stranu je potřeba umožnit interoperabilitu mezi RDF nástroji...
- ... aby mohla tvrzení být snadno sdílena, upravována a propojována
- Existuje řada definovaných formátů pro reprezentaci RDF dat (grafů)



Formáty RDF: XML

```
<?xml version="1.0"?>
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:measure="http://onto.com/measures#"
  xmlns:bi-kom-rdf="http://example.com/bi-kom/rdf/example#"
  xmlns:base="http://example.com/bi-kom/rdf/example#"
>
  <rdf:Description rdf:about="http://example.com/bi-kom/rdf/example#ema">
    <bi-kom-rdf:isMincing rdf:nodeID="a" />
    <bi-kom-rdf:hasName>Ema</bi-kom-rdf:hasName>
  </rdf:Description>

  <rdf:Description rdf:nodeID="a">
    <bi-kom-rdf:isMeat rdf:resource="http://ontology.com/meats#Beef" />
    <measure:weight rdf:datatype="http://www.w3.org/2001/XMLSchema#int">300</measure:weight>
  </rdf:Description>
</rdf:RDF>
```

Formáty RDF: Notation3 (N3), Turtle

- Notation3 je *readable RDF syntax*
- Obsahuje řadu rozšíření nad rámec RDF (speciální direktivy, syntactic sugar, ...)
- Jeho zjednodušená podmnožina (pro běžné RDF) se nazývá Turtle

```
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix bi-kom-rdf: <http://example.com/bi-kom/rdf/example#> .
@prefix measure: <http://ontol.com/measures#> .

bi-kom-rdf:ema
  bi-kom-rdf:isMincing [
    bi-kom-rdf:isMeat <http://ontology.com/meats#Beef> ;
    measure:weight "300"^^xsd:int
  ] ;
  bi-kom-rdf:hasName "Ema" .
```

Formáty RDF: TriG

- TriG je rozšířením Turtle pro reprezentaci datasetů / grafů

```
@prefix dc: <http://purl.org/dc/terms/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix bi-kom-rdf: <http://example.com/bi-kom/rdf/example#> .
@prefix measure: <http://ontol.com/measures#> .

# default graph
  {
    <http://example.org/bob> dc:publisher "Bob" .
    <http://example.org/alice> dc:publisher "Alice" .
  }

<http://example.com/bi-kom/rdf/example#Graph1>
  {
      bi-kom-rdf:ema bi-kom-rdf:hasName "Ema" .
      # ... same as in Turtle
  }
```

Formáty RDF: N-Triples

- N-Triples je podmnožina Turtle pro snadné parsování
- Obsahuje přímo jednotlivé trojice bez dalších direktiv (např. prefix)
- Může obsahovat prázdné uzly, literály a URI

```
<http://example.com/bi-kom/rdf/example#ema> <http://example.com/bi-kom/rdf/example#isMincing> _:a .

_:a <http://example.com/bi-kom/rdf/example#isMeat> <http://ontology.com/meats#Beef> .

_:a <http://ontol.com/measures#weight> "300"^^xsd:int .

<http://example.com/bi-kom/rdf/example#ema> <http://example.com/bi-kom/rdf/example#hasName> "Ema" .
```

Formáty RDF: N-Quads

- N-Quads je obdoba N-Triples ale opět pro datasety / grafy
- Místo trojic jsou zde čtveřice, kde čtvrtý prvek je identifikátor grafu

```
<http://example.com/bi-kom/rdf/example#ema> <http://example.com/bi-kom/rdf/example#isMincing> _:a <http://example.com/bi-kom/rdf/example#Graph1> .
_:a <http://example.com/bi-kom/rdf/example#isMeat> <http://ontology.com/meats#Beef> <http://example.com/bi-kom/rdf/example#Graph1> .
_:a <http://ontol.com/measures#weight> "300"^^xsd:int <http://example.com/bi-kom/rdf/example#Graph1> .
<http://example.com/bi-kom/rdf/example#ema> <http://example.com/bi-kom/rdf/example#hasName> "Ema" <http://example.com/bi-kom/rdf/example#Graph2> .
```

Formáty RDF: JSON-LD

- JSON for Linking Data využívá JSON pro reprezentaci linked data
- Speciální atributy `@context`, `@id`, `@type`, ...

```
{
  "@context": {
    "@version": 1.1,
    "name": "http://schema.org/name",
    "Person": "http://schema.org/Person",
    "Document": {
      "@id": "http://xmlns.com/foaf/0.1/Document",
      "@context": {"@vocab": "http://xmlns.com/foaf/0.1/"}
    }
  },
  "@type": "Person",
  "name": "Marek Suchánek"
}
```

RDFS: Resource Description Framework Schema

- RDF dává naprostou volnost v tom, jak trojice formovat
- To je výhoda z hlediska flexibility (dá se *ohnout* prakticky na cokoliv)
- Na druhou stranu může způsobovat řadu nejasností a konfliktů
- RDFS je jednoduchý **ontologický jazyk** patřící do rodiny RDF
- Umožňuje definovat a rozlišovat typy zdrojů (třídy) a jejich vlastnosti

RDFS: Třídy

- RDFS definuje tyto třídy:
  - `rdfs:Resource` = všechny zdroje v RDF jsou vždy implicitně členy této třídy
  - `rdfs:Class` = zdroje, které jsou samy RDF třídami
  - `rdfs:Literal` = literály v RDF dokumentech
  - `rdf:XMLLiteral` = literály používající XML syntaxi
  - `rdfs:Datatype` = datové typy
  - `rdf:Property` = vlastnosti
- `rdfs` je běžný prefix `http://www.w3.org/2000/01/rdf-schema#`

RDFS: Vlastnosti

- Třídy samotné by byly prakticky k ničemu, je potřeba mít i definované vlastnosti:
  - `rdf:type` = mezi subjektem a třídou
  - `rdfs:domain` = určení třídy subjektu
  - `rdfs:range` = určení třídy objektu
  - `rdfs:subClassOf` = mezi podtřídou a nadtřídou
  - `rdfs:subPropertyOf` = analogické k `rdfs:subClassOf` ale pro vlastnosti
  - `rdfs:label` = popis pro lidi (hodnotou je lokalizovatelný řetězec)
  - `rdfs:comment` = podrobnější popis zdroje

RDFS: Příklad

```
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix bi-kom-rdf: <http://example.com/bi-kom/rdf/example#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

bi-kom-rdf:Person rdf:type rdfs:Class .
bi-kom-rdf:Person rdfs:label "Person"@en .
bi-kom-rdf:Person rdfs:label "Osoba"@cs .

bi-kom-rdf:Woman rdf:type rdfs:Class .
bi-kom-rdf:Woman rdfs:subClassOf bi-kom-rdf:Person .

bi-kom-rdf:Man rdf:type rdfs:Class .
bi-kom-rdf:Man rdfs:subClassOf bi-kom-rdf:Person .

bi-kom-rdf:hasName rdf:type rdfs:Property .
bi-kom-rdf:hasName rdfs:domain bi-kom-rdf:Person .
bi-kom-rdf:hasName rdfs:range xsd:string .
```

RDFS: Existující příklady

- DCMS (Dublin Core Metadata Element Set)
- FOAF (Friend Of A Friend)
- vCard Ontology - for describing People and Organizations

OWL: Web Ontology Language

- Pomocí RDFS lze snadno specifikovat jednoduché ontologie: třídy a vztahy mezi nimi (případně datovými typy)
- Někdy ale není expresivita RDFS dostatečná...
- ...multiplicita, povinnost vztahu, ekvivalence tříd, *complete/disjoint* atd.
- **Web Ontology Language (OWL)** je rodina jazyků pro reprezentaci znalostí v rámci ontologií, tedy jako specifikaci typů objektů a vztahů mezi nimi
- Jazyky OWL jsou postaveny na RDF a RDFS
- Jako *OWL* často myslíme *OWL 2* (specifikace z 2009) a ne *OWL* (2004)
- *OWL* má tři varianty: OWL Lite, OWL DL a OWL Full

OWL: Lite

- Nejjednodušší varianta OWL určená primárně pro:
  - Třídní hierarchie
  - Základní omezení
- Hlavní důvod bylo usnadnění vývoje nástrojů omezujících se pouze na podporu OWL Lite
- Neumí například plnou kardinalitu ale jen 0/1, negace, disjunkci nebo enumerace
- Postupem času se zjistilo, že usnadnění vývoje není tak významné
- OWL Lite se prakticky nevyužívá, resp. jen jako subset OWL DL

OWL: DL

- Každá platná OWL Lite ontologie je také platnou OWL DL ontologií
- Název se odkazuje na *Description Logic*, z něhož čerpá své formální základy
- Klíčové vlastnosti:
  - *úplnost* = vždy tvrzení buď platí φ a nebo neplatí $\neg\varphi$
  - *rozhodnutelnost* = existuje efektivní postup, jak určit zda je tvrzení φ odvoditelné nebo není odvoditelné z množiny tvrzení
- Obsahuje všechny OWL konstrukty, ale některé jsou omezeny (nedovoluje například meta-modelování)

OWL: Full

- Každá platná OWL DL ontologie je také platnou OWL Full ontologií
- Zachovává kompatibilitu s RDFS, které nemá tak striktní omezení jako OWL DL
- Dovoluje **meta-modelování**: třída může být i sama instancí

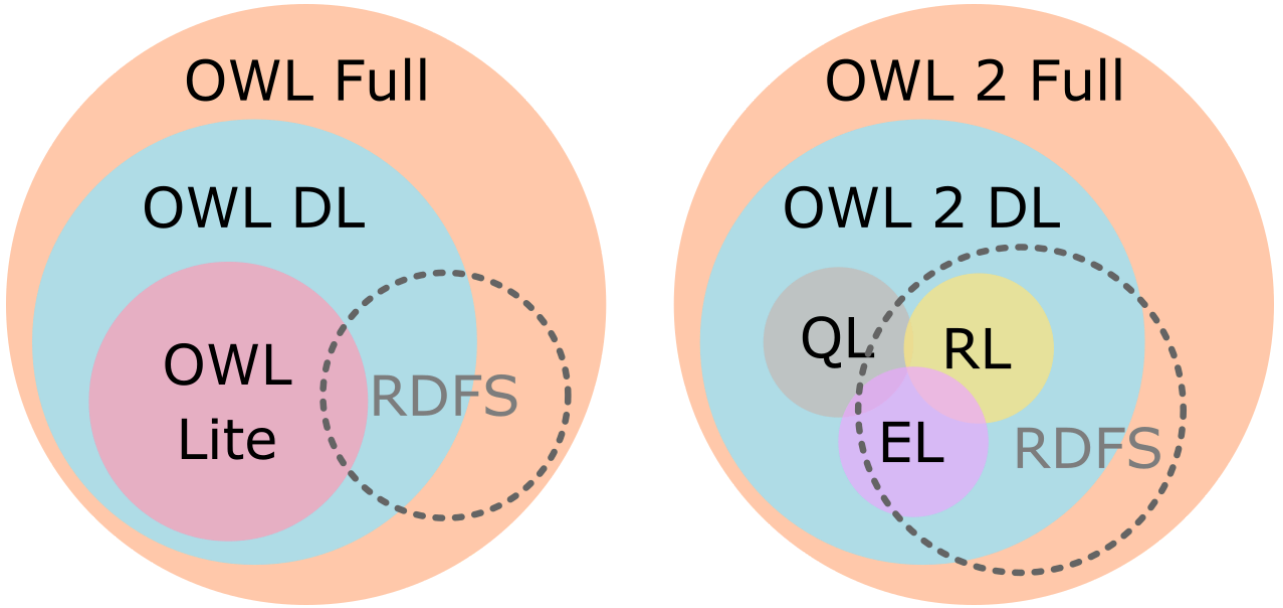
```
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

_:A rdf:type owl:Class
_:B rdf:type owl:Class
_:A rdf:type _:B
```

- Kvůli rozvolnění pravidel již není zajištěna *rozhodnutelnost*

OWL 2

- Rozdělení v OWL 2 (2009) je jiné – na *profily* (specifikace):
  - **OWL 2 EL**
  - **OWL 2 QL**
  - **OWL 2 RL**
- Tyto profily jsou podmnožiny "DL", což je podmnožina "Full"



Co OWL umí (výběr)

- Třídy, vlastnosti (*property*), domain, range, datové typy
- Rozlišuje `ObjectProperty` a `DatatypeProperty`
- Umožňuje specifikovat vlastnosti *property*:
  - *Functional* = unikátní objekt pro subjekt
  - *Irreflexive*
  - *Symmetric* / *Asymmetric*
  - *Transitive*
  - Kardinalitu: *maxCardinality*, *minCardinality*, ...

Co OWL umí (výběr)

- Mezi vztahy lze:
  - *inverseOf* = opačná vlastnost (např. `drives` a `hasDriver`)
  - *subPropertyOf* = podtyp vlastnosti
  - *equivalentProperty* = shodná vlastnost
  - *sameAs*
  - *propertyDisjointWith*, ...
- Mezi třídami lze:
  - *subClassOf*
  - *equivalentClass*
  - *sameAs*
  - *unionOf*, *intersectionOf*, ...

OWL: Příklad

```
@prefix   dc:  <http://purl.org/dc/elements/1.1> .
@prefix  owl: <http://www.w3.org/2002/07/owl#> .
@prefix  rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix     : <http://example.com/bi-kom/rdf/example#> .

<http://example.com/bi-kom/rdf/example> a owl:Ontology ;
     dc:title "Example ontology for BI-KOM course" .

:Person a owl:Class ;
        rdfs:label "Person"@en ;
        rdfs:label "Osoba"@cs .

:Woman a rdfs:Class ;
       rdfs:subClassOf :Person .

:Man a rdfs:Class ;
     rdfs:subClassOf :Person ;
     owl:disjointWith :Woman .

:Person owl:equivalentClass [ owl:disjointUnionOf ( :Woman :Man ) ] .
# ...
```

OWL: Příklad

```
# ...
:hasName rdf:type owl:DatatypeProperty ;
         rdfs:domain :Person ;
         rdfs:range xsd:string .

:ancestor rdf:type owl:ObjectProperty, owl:TransitiveProperty ;
          rdfs:domain :Person ;
          rdfs:range :Person ;
          owl:inverseOf :descendant .

_:x1 rdf:type owl:Restriction ;
     owl:onProperty :ancestor ;
     owl:maxCardinality "2"^^xsd:nonNegativeInteger .
#...
```

- Podívejte se na OWL ontologii samotného OWL: https://www.w3.org/2002/07/owl

Konceptualizace s OWL

- Pomocí OWL můžeme vytvářet konceptuální modely podobné těm z OntoUML
- Existují dokonce transformace z OntoUML i UML do OWL (i zpět)
- Pro ontologii UFO existuje gUFO (OWL 2 DL reprezentace)
- Obdobně existují i další *základní* a *doménové* ontologie pro tvorbu konceptualizací
- Stačí je použít jako slovník pro tvorbu svých tříd a vlastností
- Každá ontologie by měla mít vlastní metadata (název, popis, verzi, autory) a jakožto RDF zdroj i URI, kde by měly být dokumentace:
  - https://nemo-ufes.github.io/gufo/
  - https://nemo-ufes.github.io/gufo/#RoleMixin

Konceptualizace s OWL a gUFO

```
@prefix  owl: <http://www.w3.org/2002/07/owl#> .
@prefix  rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix gufo: <http://purl.org/nemo/gufo#> .
@prefix     : <http://purl.org/nemo/example#> .

:Person a owl:Class ;
        rdfs:subClassOf gufo:Object ;
        rdf:type gufo:Kind .

:olderThan a owl:ObjectProperty ;
           rdfs:subPropertyOf gufo:ComparativeRelationshipType ;
           rdfs:domain :Person ;
           rdfs:range :Person .
```

Zapisujeme sémanticky to samé, co v OntoUML, ale *jiným jazykem*.

Pozn.: Vygenerovat gUFO z vašeho OntoUML modelu a vysvětlit, v čem se liší od vlastního ručně psaného OWL, je **bonusový úkol** semestrální práce.

SPARQL: Protocol and RDF Query Language

- Sémantický dotazovací jazyk pro data uchovaná ve formátu RDF
- Jedna z klíčových technologií sémantického webu
- Na první pohled podstatná podobnost s SQL
- Dotazování funguje na základě shody v subject, predicate a object
- Typy dotazů:
  - `SELECT` = výběr dat ve formě tabulky dle dotazu
  - `CONSTRUCT` = výběr dat ve formě RDF dle dotazu
  - `ASK` = vrací pravdivostní hodnotu `true` nebo `false`
  - `DESCRIBE` = vrací popis dat vyhovujících dotazu jako RDF

SPARQL: SELECT

```
PREFIX bi-kom-rdf: <http://example.com/bi-kom/rdf/example#>
PREFIX measure: <http://onto.com/measures#>

SELECT ?name ?weight
# FROM (dataset definition)
WHERE {
   ?person a bi-kom-rdf:Person ;
           bi-kom-rdf:isMincing ?meat ;
           bi-kom-rdf:hasName ?name .
   ?meat measure:weight ?weight .
   FILTER (?weight > 100)
}
ORDER BY DESC(?weight)
LIMIT 10
```

SPARQL: `CONSTRUCT`

- Vytvoř triplety se jménem podle Friend-of-a-Friend, ale jen pro muže

```
PREFIX bi-kom-rdf: <http://example.com/bi-kom/rdf/example#>
PREFIX foaf: <http://xmlns.com/foaf/spec/>

CONSTRUCT { ?man foaf:name ?name }
WHERE {
  ?man a bi-kom-rdf:Man ;
       bi-kom-rdf:hasName ?name .
}
```

SPARQL: `ASK`

- Existuje žena, která mele maso?

```
PREFIX bi-kom-rdf: <http://example.com/bi-kom/rdf/example#>

ASK {
   ?woman a bi-kom-rdf:Woman ;
          bi-kom-rdf:isMincing ?meat .
   ?meat a bi-kom-rdf:Meat .
}
```

SPARQL: `DESCRIBE`

- Popiš osobu (nebo osoby), která se jmenuje *Ema*

```
PREFIX bi-kom-rdf: <http://example.com/bi-kom/rdf/example#>

DESCRIBE ?ema
WHERE {
  ?ema a bi-kom-rdf:Person ;
       bi-kom-rdf:hasName "Ema" .
}
```

SPARQL: Protocol and RDF Query Language

- SPARQL umožňuje i úpravy dat a to včetně čtveřic (pojmenovaných grafů)
  - Aktualizace grafu: `INSERT DATA`, `DELETE DATA`, `DELETE`, `INSERT`, `LOAD`, `CLEAR`
  - Práce s grafem: `CREATE`, `DROP`, `COPY`, `MOVE`, `ADD`
- Ale umí toho opět mnohem víc: `OPTIONAL`, `FILTER`, `UNION`, `DISTINCT`, `COUNT`, ...
- Rovněž lze také spouštět dotazy nad vícero SPARQL endpointy = **Federated Query** pomocí `SERVICE` (SPARQL 1.1)

SPARQL nad reálnými daty: Wikidata

- Veřejný endpoint query.wikidata.org, bez registrace — vyzkoušejte hned teď
- *Všechny vysoké školy v Česku s datem založení a městem:*

```
SELECT ?university ?universityLabel ?inception ?cityLabel WHERE {
  ?university wdt:P31/wdt:P279* wd:Q3918 ;   # instance (podtřídy) vysoké školy
              wdt:P17 wd:Q213 .              # země = Česko
  OPTIONAL { ?university wdt:P571 ?inception . }
  OPTIONAL { ?university wdt:P131 ?city . }
  SERVICE wikibase:label { bd:serviceParam wikibase:language "cs,en". }
}
ORDER BY ?inception
```

- Nikdo pro tuto otázku nenavrhoval tabulku a nikoho nebylo potřeba žádat o export

SHACL: Shapes Constraint Language

- RDFS a OWL říkají, co lze **odvodit**, ne co data **musí obsahovat**
  - `rdfs:range xsd:string` u `:hasName` neznamená, že každá osoba má jméno
  - V otevřeném světě není chybějící tvrzení chyba, je *neznámé*
- Aplikace ale potřebuje vědět: *jsou tato data dost dobrá na to, abych je zpracoval?*
- SHACL (W3C Recommendation, 2017) validuje RDF graf proti **shapes**
- Shape říká: *uzly tohoto druhu musí mít tyto vlastnosti s těmito hodnotami*
- Shapes se samy zapisují v RDF (Turtle), takže není potřeba se učit novou syntaxi

SHACL vs. OWL

- **OWL** pracuje v **otevřeném světě**:
  - Chybějící tvrzení je *neznámé* a může být odvozeno později
  - Osoba bez jména je zcela v pořádku
  - Účel: význam a **odvozování**
- **SHACL** pracuje v **uzavřeném světě** daného grafu:
  - Chybějící tvrzení je **porušení**, které je potřeba nahlásit a opravit
  - Osoba bez jména neprojde validací
  - Účel: **kvalita dat** a validace

- **Nejsou** to alternativy — odpovídají na jiné otázky a používají se společně
- SHACL je pro RDF data zhruba tím, čím je **OCL** pro UML/OntoUML model

SHACL: Příklad

```
@prefix  sh: <http://www.w3.org/ns/shacl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix    : <http://example.com/bi-kom/rdf/example#> .

:PersonShape a sh:NodeShape ;
    sh:targetClass :Person ;          # co validovat
    sh:property [
        sh:path :hasName ;            # ... a na které cestě
        sh:minCount 1 ;
        sh:maxCount 1 ;
        sh:datatype xsd:string ;
        sh:message "Osoba musí mít právě jedno jméno."@cs ;
    ] ;
    sh:property [
        sh:path :ancestor ;
        sh:maxCount 2 ;
        sh:class :Person ;
    ] .
```

SHACL: Protokol o validaci

- Výsledkem validace je opět **RDF**, lze se na něj tedy dotazovat, ukládat jej i sdílet

```
[] a sh:ValidationReport ;
   sh:conforms false ;
   sh:result [
       a sh:ValidationResult ;
       sh:resultSeverity sh:Violation ;
       sh:focusNode :ema ;
       sh:resultPath :hasName ;
       sh:sourceConstraintComponent sh:MinCountConstraintComponent ;
       sh:resultMessage "Osoba musí mít právě jedno jméno."@cs ;
   ] .
```

- Závažnost může být i `sh:Warning` nebo `sh:Info`, nejen `sh:Violation`
- Běžná omezení: `minCount`, `maxCount`, `datatype`, `class`, `nodeKind`, `pattern`, `in`, `languageIn`, `uniqueLang`, `lessThan`, `node`, …

Totéž "2" ve třech jazycích

- *Osoba má nejvýše dva předky.* Stejná věta, tři různé významy:

```
_:x1 rdf:type owl:Restriction ;                    # OWL: existují nejvýše 2 předci
     owl:onProperty :ancestor ;                    # třetí není chyba - reasoner odvodí,
     owl:maxCardinality "2"^^xsd:nonNegativeInteger .   # že dva z nich jsou tentýž!
```

```
context Person inv:                                -- OCL: nad uzavřeným UML modelem
  self.ancestor->size() <= 2
```

```
:PersonShape a sh:NodeShape ;                      # SHACL: nad konkrétními daty
    sh:targetClass :Person ;                       # třetí je nahlášen jako porušení
    sh:property [ sh:path :ancestor ; sh:maxCount 2 ] .
```

Grafové databáze

- Nad čím spouštět SPARQL dotazy? Kam ukládat RDF?
- Řešením jsou grafové databáze (nebo také *triple stores*), které ukládají trojice (nebo čtveřice) typicky do různých *repository* a poskytují doplňující funkce (import, export, přehledy, SPARQL endpoint, uživatelské rozhraní, účty a práva, ...)
  - GraphDB
  - AllegroGraph
  - BlazeGraph
  - Virtuoso
  - Neo4j
  - InfiniteGraph

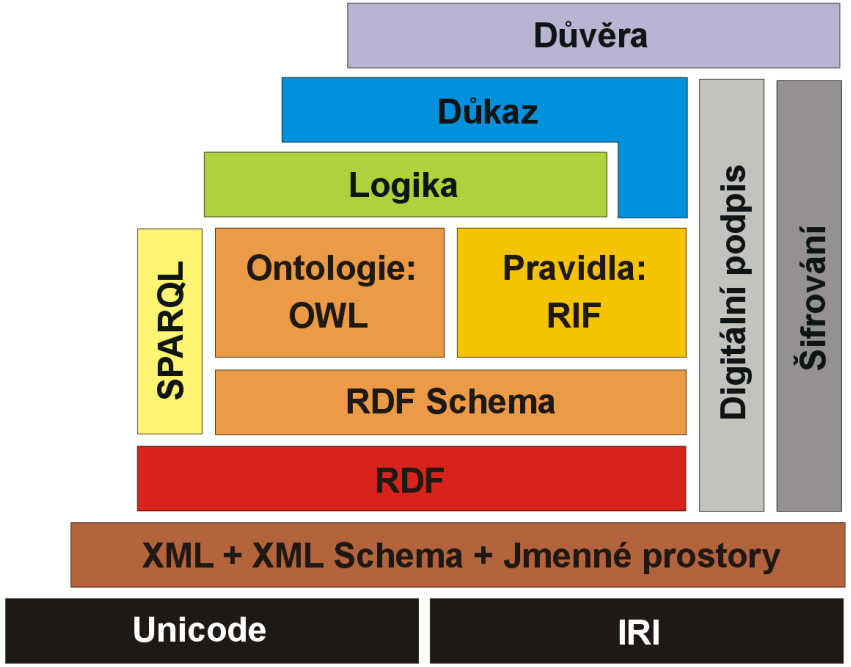
(a další... více na NI-PDB)

Nástroje pro práci s RDF/OWL

- Existuje řada nástrojů pro práci s RDF/OWL:
  - Modelování ontologií: Protégé
  - Vizualizace: WebVOWL, OWLGrEd
  - Dokumentace: WIDOCO, pyLODE
- Knihovny pro práci s RDF/OWL:
  - Java: Apache Jena, rdf4j, Apache Commons RDF, ...
  - Python: rdflib
  - .NET: dotNetRDF
  - ...

Sémantický web a Schema.org

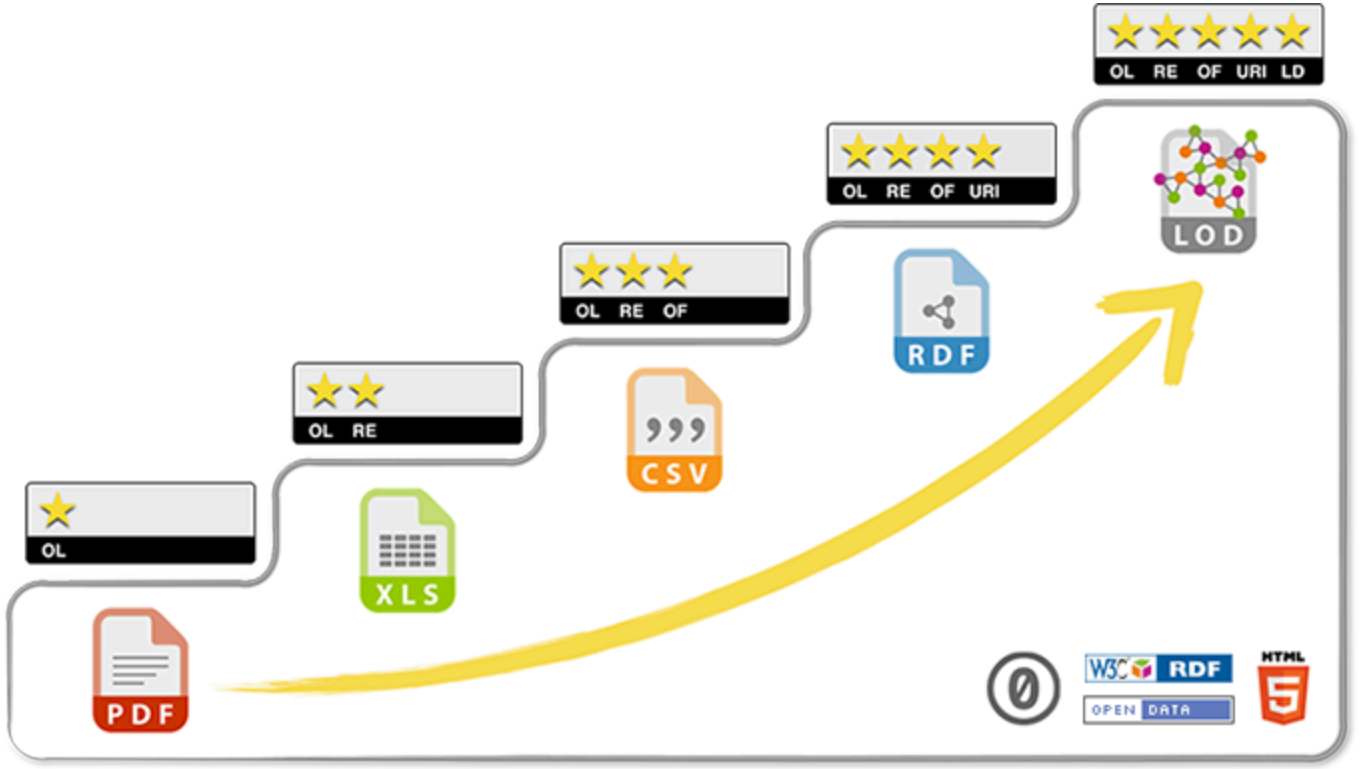
- Sémantický web je postavený na XML, RDF a URI
- Základem je **konceptualizace dat** dostupných na Internetu pomocí ontologií
- **Mikroformáty** slouží pro značkování strojově čitelných informací v (X)HTML



(Příkladem je značkování pomocí slovníku Schema.org; více na NI-SWE)

5★ otevřená data

- Tim Berners-Lee navrhl 5-hvězdičkové schéma nasazení pro otevřená data

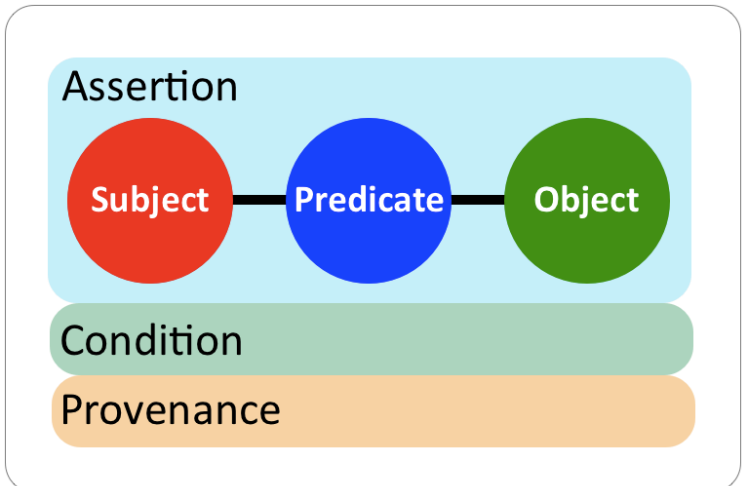


FAIR principy a URI zdroje

- Principy FAIR (meta)dat
  - Findable = (meta)data mají persistentní identifikátor, metadata jsou vždy dostupná
  - Accessible = (meta)data jsou přístupná standardními protokoly
  - Interoperable = (meta)data jsou ve standardizovaných formátech
  - Re-usable = (meta)data mají jasné licence pro znovupoužití
- URI zdroje v RDF by mělo být *resolvable* (na adrese by měl opravdu zdroj existovat) a současně URI by mělo být persistentní:
  - pURL.org
  - w3id.org
  - ...

Nanopublikace

- Nanopublikace jsou minimální jednotkou publikovatelné informace
- Projekt využívá technologie RDF pro zachycení publikací včetně metadata jako FAIR data
- Související nástroje umožňují mimo jiné:
  - tvorbu distribuovaných úložišť pro nanopublikace
  - vyhledávání a filtrování
  - vytváření a podepisování nanopublikací dle šablon



Semestrální práce: část s RDF (10 bodů)

- **5 bodů** — alespoň **10 tříd vašeho OntoUML modelu** (včetně vlastností) v RDFS/OWL a instanční dataset o alespoň **100 tripletech** (`ontology.ttl` + `data.ttl`)
  - Bonus **+1 bod** za vyjádření modelu pomocí **gUFO**
- **3 body** — **3 SPARQL dotazy** (`q1.rq`, `q2.rq`, `q3.rq`)
- **2 body** — **2 SHACL omezení** (`shape1.ttl`, `shape2.ttl`)
  - Jedno z nich musí zachycovat **OCL omezení z 1. části** semestrální práce
- Dataset **může být vygenerovaný AI** — pořád je ale váš, musíte jej zkontrolovat a obhájit
- Podrobnosti a hodnocení: cvičení #9 a BI-KOM: Semestrální práce

Kdy se RDF vyplatí?

- Technologie RDF/OWL se **vyplatí**, když:
  - Data pochází z **mnoha zdrojů** s různými schématy a je potřeba je propojit
  - Doménový model se **často mění** nebo není dopředu úplně známý
  - Data se **publikují** pro neznámé konzumenty a další využití (otevřená data, FAIR)
  - Význam, původ a metadata jsou stejně důležité jako samotné hodnoty
  - Otázky mají tvar grafu: cesty, hierarchie, *kdo je s kým propojený*
- Technologie RDF/OWL jsou spíše **překážkou**, když:
  - Jde o uzavřenou, stabilní a transakčně náročnou zátěž (objednávky, bankovnictví)
  - Jeden tým vlastní jedno schéma jedné aplikace (relační databáze je jednodušší)
  - Nikdo v týmu s tím nemá zkušenost — vstupní cena je reálná

Doporučení na závěr

- Technologie RDF/OWL jsou velmi univerzální a lze je uplatnit v celé řadě případů
- Tyto technologie jsou používané jak v akademické, tak i v komerční sféře
- Stejně jako u ostatních je potřeba si před použitím zvážit pro a proti:
  - Rychlost a efektivita využití zdrojů
  - Dostupnost dokumentace, velikost komunity
  - Podpora a udržovanost ("mrtvé" knihovny a nástroje)
  - Stabilita a spolehlivost
  - ...
- Někdy mohou RDF/OWL technologie výrazně pomoci, jindy ale mohou klást překážky ve vývoji a vést k depresi vývojářů

Časté otázky

„Proč OWL nehlásí chybu, když chybí povinný údaj?“

V otevřeném světě chybějící tvrzení samo o sobě neznamená, že daná skutečnost neexistuje. OWL může vyžadovat existenci hodnoty, aniž by byla explicitně uvedena v datech. Požadavek na její přítomnost v posuzovaném grafu vyjádříme vhodným SHACL pravidlem.

Časté otázky

„Když SPARQL dotaz nic nevrátí, znamená to, že taková věc neexistuje?“

Znamená to, že dotaz v daném datasetu a při daném způsobu vyhodnocení nenašel shodu.

Příčinou mohou být chybějící data, jiný pojmenovaný graf nebo neprovedené odvození.

Závěr o neexistenci potřebuje další předpoklady o úplnosti evidence.

Časté otázky

„Když dvě organizace používají RDF, rozumějí si jejich data automaticky?“

RDF poskytuje společný způsob zápisu a propojování tvrzení. Stále však musíme sladit význam pojmů: „zákazník“ může v každé organizaci označovat jinou skupinu. Technické propojení samo nezaručuje sémantickou shodu.

Konec přednášky